ODAT-SE SIM-TRHEPD-RHEED Module Documentation

リリース *1.1.0*

ISSP, University of Tokyo

2026 年 10 月 10 日

Contents:

第 1 章　はじめに **1**
1.1 ODAT-SE とは . . . 1
1.2 odatse-STR とは . . . 1
1.3 ライセンス . . . 1
1.4 バージョン履歴 . . . 2
1.5 主な開発者 . . . 2

第 2 章　odatse-STR のインストール **5**
2.1 実行環境・必要なパッケージ . . . 5
2.2 ダウンロード・インストール . . . 5
2.3 実行方法 . . . 6
2.4 アンインストール . . . 6

第 3 章　チュートリアル **7**
3.1 TRHEPD 順問題ソルバー . . . 7
3.2 Nelder-Mead 法による最適化 . . . 12
3.3 グリッド型探索 . . . 17
3.4 ベイズ最適化 . . . 22
3.5 レプリカ交換モンテカルロ法による探索 . . . 29
3.6 ポピュレーションアニーリングによる探索 . . . 33
3.7 ユーザープログラムによる解析 . . . 38

第 4 章　入出力 **45**
4.1 入力パラメータ . . . 45
4.2 ソルバー用補助ファイル . . . 48

第 5 章　謝辞 **53**

第 6 章　お問い合わせ **55**

第1章　はじめに

1.1 ODAT-SE とは

ODAT-SE は、順問題ソルバーに対して探索アルゴリズムを適用して最適解を探すためのフレームワークです。順問題ソルバーはユーザー自身で定義できるほか、標準的な順問題ソルバーとして 2 次元物質構造解析向け実験データ解析ソフトウェアが用意されています。順問題ソルバーでは、原子位置などをパラメータとして得られたデータと実験データとのずれを損失関数として与えます。探索アルゴリズムによりこの損失関数を最小化する最適なパラメータを推定します。現バージョンでは、順問題ソルバーとして量子ビーム回折実験の全反射高速陽電子回折法 (Total-Reflection High-Energy Positron Diffraction: TRHEPD), 表面 X 線回折法 (Surface X-ray Diffraction: SXRD), 低速電子線回折法 (Low Energy Electron Diffraction: LEED) に対応しており、探索アルゴリズムは Nelder-Mead 法, グリッド型探索法, ベイズ最適化, レプリカ交換モンテカルロ法, ポピュレーションアニーリングモンテカルロ法が実装されています。

1.2 odatse-STR とは

全反射高速陽電子回折法 (TRHEPD) および反射高速電子線回折法 (Reflection High Energy Electron Diffraction: RHEED) のシミュレータソフトウェアとして sim-trhepd-rheed が開発されています。原子位置を入力として回折強度の Rocking Curve を計算するプログラムで、Fortran で書かれており、標準的な Linux 環境で動作します。odatse-STR は、この sim-trhepd-rheed を ODAT-SE の順問題ソルバとして利用するためのアダプタライブラリです。2DMAT v2.x の順問題ソルバーの一つとして開発されたコンポーネントを、独立なモジュールとして再構成したものです。ODAT-SE および sim-trhepd-rheed と組み合わせて使用します。

1.3 ライセンス

本ソフトウェアのプログラムパッケージおよびソースコード一式は GNU General Public License version 3 (GPL v3) に準じて配布されています。

Copyright (c) <2020-> The University of Tokyo. All rights reserved.

本ソフトウェアは 2020 年度および 2024 年度 東京大学物性研究所 ソフトウェア高度化プロジェクトの支援を受け開発されました。ODAT-SE を引用する際には以下の文献を引用してください。

"Data-analysis software framework 2DMAT and its application to experimental measurements for two-dimensional material structures", Y. Motoyama, K. Yoshimi, I. Mochizuki, H. Iwamoto, H. Ichinose, and T. Hoshi, Computer Physics Communications 280, 108465 (2022).

Bibtex:

```
@article{MOTOYAMA2022108465,
  title = {Data-analysis software framework 2DMAT and its application to experimental␣
↪measurements for two-dimensional material structures},
  journal = {Computer Physics Communications},
  volume = {280},
  pages = {108465},
  year = {2022},
  issn = {0010-4655},
  doi = {https://doi.org/10.1016/j.cpc.2022.108465},
  url = {https://www.sciencedirect.com/science/article/pii/S0010465522001849},
  author = {Yuichi Motoyama and Kazuyoshi Yoshimi and Izumi Mochizuki and Harumichi␣
↪Iwamoto and Hayato Ichinose and Takeo Hoshi}
}
```

1.4 バージョン履歴

odatse-STR

- v1.1.0 : 2026-10-09
- v1.0.1 : 2026-10-09
- v1.0.0 : 2024-11-25

ODAT-SE

- v3.0.0 : 2024-11-25

2DMAT

- v2.1.0 : 2022-04-08
- v2.0.0 : 2022-01-17
- v1.0.1 : 2021-04-15
- v1.0.0 : 2021-03-12
- v0.1.0 : 2021-02-08

1.5 主な開発者

ODAT-SE, odatse-STR, 2DMAT は以下のメンバーで開発しています.

- ODAT-SE v3.0.0 -, odatse-STR v1.0.0 -
  - 本山 裕一 (東京大学 物性研究所)
  - 吉見 一慶 (東京大学 物性研究所)
  - 青山 龍美 (東京大学 物性研究所)
  - 星 健夫 (核融合科学研究所)

- 2DMAT v2.0.0 -
  - 本山 裕一 (東京大学 物性研究所)
  - 吉見 一慶 (東京大学 物性研究所)
  - 岩本 晴道 (鳥取大学 大学院工学研究科)
  - 星 健夫 (鳥取大学 大学院工学研究科)
- 2DMAT v0.1.0 -
  - 本山 裕一 (東京大学 物性研究所)
  - 吉見 一慶 (東京大学 物性研究所)
  - 星 健夫 (鳥取大学 大学院工学研究科)

第2章　odatse-STR のインストール

2.1 実行環境・必要なパッケージ

- python 3.9 以上
  - 必要な python パッケージ
    - tomli (>= 1.2)
    - numpy (>= 1.14)
- ODAT-SE version 4.0 以降
- sim-trhepd-rheed version 1.0.2 以降

2.2 ダウンロード・インストール

1. ODAT-SE をインストールする
   - ソースコードからのインストール

     リポジトリから ODAT-SE のソースファイルを取得します。

     ```
     $ git clone https://github.com/issp-center-dev/ODAT-SE.git
     ```

     pip コマンドを実行してインストールします。

     ```
     $ cd ODAT-SE
     $ python3 -m pip install .
     ```

     `--user` オプションを付けるとローカル (`$HOME/.local`) にインストールできます。

     `python3 -m pip install .[all]` を実行するとオプションのパッケージも同時にインストールします。

2. sim-trhepd-rheed をインストールする
   - 公式サイトからソースパッケージをダウンロードします。

     https://github.com/sim-trhepd-rheed/sim-trhepd-rheed/releases/tag/v1.0.2

     ```
     $ wget -O sim-trhepd-rheed-1.0.2.tar.gz https://github.com/sim-trhepd-rheed/
     ↪sim-trhepd-rheed/archive/refs/tags/v1.0.2.tar.gz
     ```

- ソースパッケージを展開し、コンパイルします。必要に応じて sim-trhepd-rheed/src 内の Makefile を編集してください。

```
$ tar xvfz sim-trhepd-rheed-1.0.2.tar.gz
$ cd sim-trhepd-rheed-1.0.2/src
$ make
```

実行形式 `bulk.exe`, `surf.exe`, `potcalc.exe`, `xyz.exe` が作成されます。`bulk.exe` と `surf.exe` を PATH の通ったディレクトリ (環境変数 PATH に列挙された、実行プログラムを探索するディレクトリ) に配置するか、実行時にディレクトリ名をつけて指定します。

3. odatse-STR をインストールする

- ソースコードからのインストール

odatse-STR のソースファイルは GitHub リポジトリから取得できます。以下の手順でリポジトリをクローンした後、pip コマンドを実行してインストールします。

```
$ git clone https://github.com/2DMAT/odatse-STR.git
$ cd odatse-STR
$ python3 -m pip install .
```

`--user` オプションを付けるとローカル (`$HOME/.local`) にインストールできます。

odatse-STR のライブラリと、実行コマンド `odatse-STR` がインストールされます。

2.3 実行方法

ODAT-SE では順問題ソルバと逆問題解析アルゴリズムを組み合わせて解析を行います。TRHEPD の解析を行うには次の 2 通りの方法があります。

1. このパッケージに含まれる odatse-STR プログラムを利用して解析を行います。ユーザは、プログラムの入力となるパラメータファイルを TOML 形式で作成し、プログラムの引数に指定してコマンドを実行します。逆問題解析のアルゴリズムはパラメータで選択できます。
2. odatse-STR ライブラリと ODAT-SE フレームワークを用いてプログラムを作成し、解析を行います。逆問題解析アルゴリズムは import するモジュールで選択します。プログラム中に入力データの生成を組み込むなど、柔軟な使い方ができます。

パラメータの種類やライブラリの利用方法については以降の章で説明します。

2.4 アンインストール

odatse-STR モジュールおよび ODAT-SE モジュールをアンインストールするには、以下のコマンドを実行します。

```
$ python3 -m pip uninstall odatse-STR ODAT-SE
```

第3章　チュートリアル

順問題ソルバーとして用意されている sim_trhepd_rheed は、東北大学の花田貴先生によって開発された全反射高速 (陽) 電子回折 (RHEED, TRHEPD) の解析ソフトウェアをベースに作成されています。

TRHEPD では、与えられた原子位置に対して回折データのシミュレーションを行います。これを原子位置から回折データへの順問題としたとき、実験で得られた回折データを再現するように原子位置を推定する逆問題を考えます。ODAT-SE では、逆問題を解くためのアルゴリズムとして、以下の 5 つのアルゴリズムが用意されています。

- minsearch

  Nelder-Mead 法

- mapper_mpi

  与えられたパラメータの探索グリッドを全探索

- bayes

  ベイズ最適化

- exchange

  レプリカ交換モンテカルロ法

- pamc

  ポピュレーションアニーリング法

本チュートリアルでは、最初に順問題プログラム sim_trhepd_rheed の実行方法を説明した後、minsearch, mapper_mpi, bayes, exchange, pamc による逆問題解析の実行方法について順に説明します。以下では odatse-STR に付属の odatse-STR プログラムを利用し、TOML 形式のパラメータファイルを入力として解析を実行します。

チュートリアルの最後に、ユーザープログラムの項で、メインプログラムを自分で作成して使う方法を minsearch を例に説明します。

3.1 TRHEPD 順問題ソルバー

odatse-STR モジュールは、反射高速 (陽) 電子回折 (RHEED, TRHEPD) の強度計算 (A. Ichimiya, Jpn. J. Appl. Phys. 22, 176 (1983); 24, 1365 (1985)) を行うプログラム sim-trhepd-rheed を ODAT-SE の順問題ソルバーとして利用するためのラッパーです。本チュートリアルでは sim-trhepd-rheed を用い、様々なアルゴリズムを利用した解析を行います。最初に、チュートリアルを行うために必要な sim-trhepd-rheed のインストールおよびテストを行います。

3.1.1 ダウンロード・インストール

あらかじめ odatse-STR のソースコードをリポジトリから取得しておきます。

```
$ git clone https://github.com/2DMAT/odatse-STR.git
$ cd odatse-STR
```

GitHub の sim-trhepd-rheed リポジトリから、ソースコード一式を入手し、ビルドします。

```
$ git clone http://github.com/sim-trhepd-rheed/sim-trhepd-rheed
$ cd sim-trhepd-rheed/src
$ make
```

Makefile は実行環境に応じて適宜編集してください。make が成功すると、 bulk.exe と surf.exe が作成されます。

3.1.2 計算実行

sim-trhepd-rheed では、最初に bulk.exe で表面構造のバルク部分に関する計算をします。その後、bulk.exe の計算結果 (bulkP.b ファイル) を用いて、 surf.exe 表面構造の表面部分を計算します。

このチュートリアルでは実際に TRHEPD 計算をしてみます。サンプルとなる入力ファイルは odatse-STR の sample/solver 以下にあります。まず、このフォルダを適当な作業用フォルダ work にコピーします。

```
$ cp -r sample/solver work
$ cd work
```

次に bulk.exe と surf.exe を work にコピーします

```
$ cp ../sim-trhepd-rheed/src/bulk.exe .
$ cp ../sim-trhepd-rheed/src/surf.exe .
```

bulk.exe を実行します。

```
$ ./bulk.exe
```

バルクファイル bulkP.b が生成されます。

```
$ ls
bulk.exe bulk.txt bulkP.b surf.exe surf.txt
```

続いて、 surf.exe を実行します。

```
$ ./surf.exe
```

以下のように出力されます。

```
surf-bulkP.s
```

実行後に、ファイル surf-bulkP.md, surf-bulkP.s および SURF YYYYMMDD-HHMMSS log.txt が生成されます。(YYYYMMDD、 HHMMSS には実行日時に対応した数字が入ります)

3.1.3 計算結果の可視化

surf-bulkP.s は以下の通りです。

```
 #azimuths,g-angles,beams
 1 56 13
 #ih,ik
 deg,6 0,5 0,4 0,3 0,2 0,1 0,0 0,-1 0,-2 0,-3 0,-4 0,-5 0,-6 0,
   5.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  1.5946E-02,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  0.0000E+00,
   6.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  1.8701E-02,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  0.0000E+00,
   7.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  2.1202E-02,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  0.0000E+00,
   8.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  2.
↪1711E-03,  1.9273E-02,  2.1711E-03,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  0.0000E+00,
   9.0000E-01,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  4.
↪3965E-03,  1.7006E-02,  4.3965E-03,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  0.0000E+00,
   1.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.0000E+00,  6.
↪3263E-03,  1.4952E-02,  6.3263E-03,  0.0000E+00,  0.0000E+00,  0.0000E+00,  0.
↪0000E+00,  0.0000E+00,
(以下略)
```

上記ファイルより、縦軸に角度 (5 行目以降の 1 列目データ) と $(0,0)$ ピークの強度 (5 行目以降の 8 列目データ) からロッキングカーブを作成します。Gnuplot 等のグラフソフトを用いる事も出来ますが、ここでは script フォルダにあるプログラム plot_bulkP.py を用います。以下のように実行して下さい。

```
$ python3 ../script/plot_bulkP.py
```

以下のような plot_bulkP.png が作成されます。

この 00 ピークの回折強度のデータに対し、コンボリューションを掛けたうえで規格化します。surf-bulkP.s があることを確認し、 make_convolution.py を実行してください。

```
$ python3 ../script/make_convolution.py
```

実行すると、以下のようなファイル convolution.txt が出力されます。

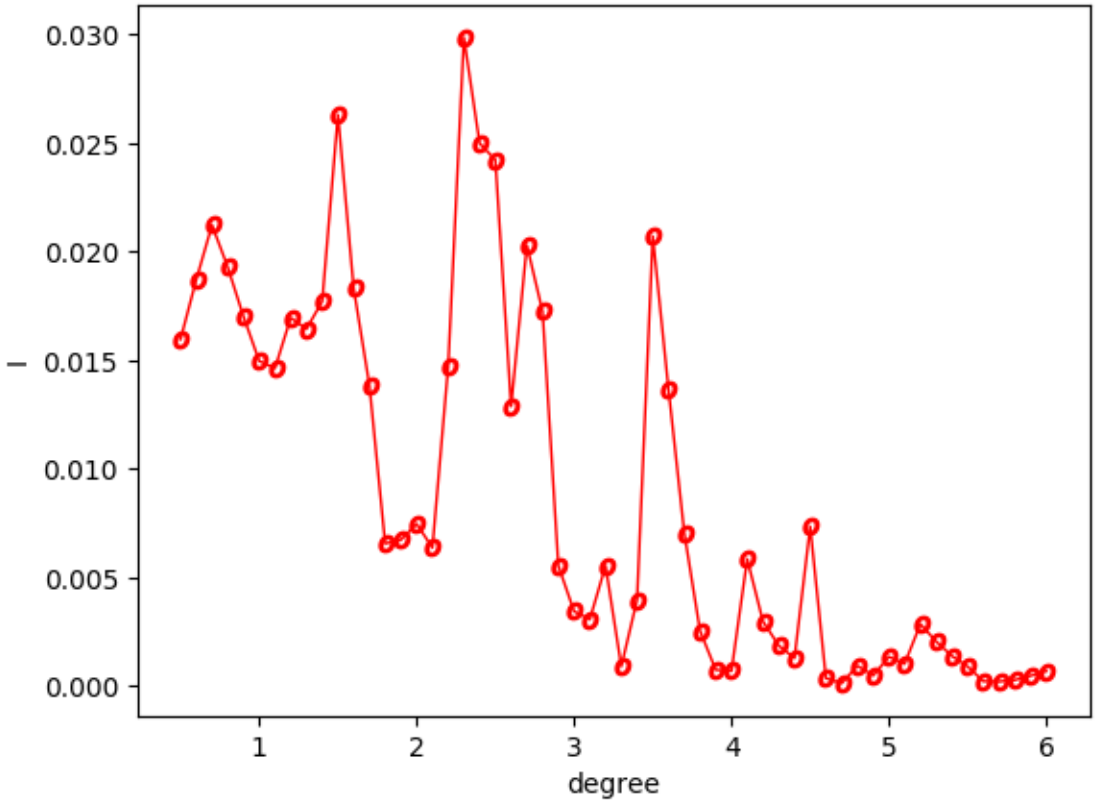


図 1: Si(001)-2x1 面のロッキングカーブ。

```
0.500000 0.010818010
0.600000 0.013986716
0.700000 0.016119093
0.800000 0.017039022
0.900000 0.017084666
(中略)
5.600000 0.000728539
5.700000 0.000530758
5.800000 0.000412908
5.900000 0.000341740
6.000000 0.000277553
```

1 列目が視射角、2 列目が `surf-bulkP.s` に書かれた 00 ピーク回折強度のデータに半値幅 0.5 のコンボリューションを付加して規格化したものです。

`plot_convolution.py` を実行すると `plot_convolution.png` が作成されます。

```
$ python3 ../script/plot_convolution.py
```

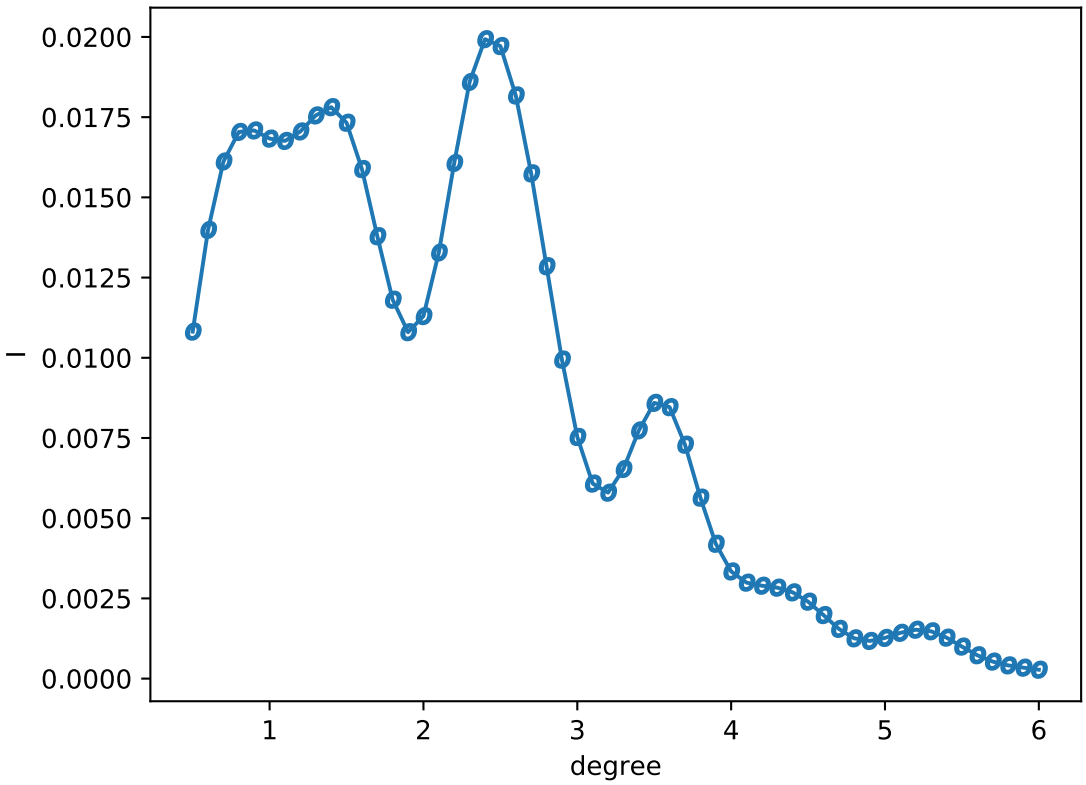


図 2: Si(001)-2x1 面のロッキングカーブに半値幅 0.5 のコンボリューションを付加して規格化したもの。

3.2 Nelder-Mead 法による最適化

ここでは、Nelder-Mead 法を用いて回折データから原子座標を解析する逆問題の計算を行う方法について説明します。具体的な計算手順は以下の通りです。

0. 参照ファイルの準備

   合わせたい参照ファイル (今回は後述する `experiment.txt` に相当) を準備する。

1. 表面構造のバルク部分に関する計算実行

   `bulk.exe` を `sample/minsearch` にコピーして計算を実行する。

2. メインプログラムの実行

   `odatse-STR` を用いて計算を実行し原子座標を推定する。

メインプログラムでは、Nelder-Mead 法 (scipy.optimize.minimize を使用) を用いて、ソルバー (今回は `surf.exe`) を用いて得られた強度と、参照ファイル (`experiment.txt`) に記載された強度のずれ (R 値) を最小化するパラメータを探索します。

3.2.1 サンプルファイルの場所

サンプルファイルは `sample/single_beam/minsearch` にあります。フォルダには以下のファイルが格納されています。

- `bulk.txt`

  `bulk.exe` の入力ファイル

- `experiment.txt` , `template.txt`

  メインプログラムでの計算を進めるための参照ファイル

- `ref.txt`

  本チュートリアルで求めたい回答が記載されたファイル

- `input.toml`

  メインプログラムの入力ファイル

- `prepare.sh` , `do.sh`

  本チュートリアルを一括計算するために準備されたスクリプト

以下、これらのファイルについて説明したあと、実際の計算結果を紹介します。

3.2.2 参照ファイルの説明

`template.txt` は `surf.exe` の入力ファイルとほぼ同じ形式のファイルです。動かすパラメータ (求めたい原子座標などの値) を「 `value_*` 」などの適当な文字列に書き換えています。以下が `template.txt` の中身です。

```
2                                   ,NELMS,  -------- Ge(001)-c4x2
32,1.0,0.1                          ,Ge Z,da1,sap
0.6,0.6,0.6                         ,BH(I),BK(I),BZ(I)
32,1.0,0.1                          ,Ge Z,da1,sap
0.4,0.4,0.4                         ,BH(I),BK(I),BZ(I)
9,4,0,0,2, 2.0,-0.5,0.5              ,NSGS,msa,msb,nsa,nsb,dthick,DXS,DYS
8                                   ,NATM
1, 1.0, 1.34502591  1       value_01   ,IELM(I),ocr(I),X(I),Y(I),Z(I)
1, 1.0, 0.752457792 1       value_02
2, 1.0, 1.480003343 1.465005851     value_03
2, 1.0, 2   1.497500418     2.281675
2, 1.0, 1   1.5     1.991675
2, 1.0, 0   1       0.847225
2, 1.0, 2   1       0.807225
2, 1.0, 1.009998328 1       0.597225
1,1                                 ,(WDOM,I=1,NDOM)
```

今回の入力ファイルでは、value_01, value_02, value_03 を用いています。サンプルフォルダには、原子位置が正しく推定できているかを知るための参照ファイルとして、ref.txt が置いてあります。ファイルの中身は

```
fx = 8.97501627575891e-06
z1 = 5.230573154947887
z2 = 4.3706085364407095
z3 = 3.5961303342247577
```

となっており、value_01, value_02, value_03 がそれぞれ z1, z2, z3 に対応しています。fx は目的関数の最適値です。experiment.txt は、メインプログラムで参照に用いるファイルです。ここではテストデータとして、template.txt に ref.txt のデータを代入し、順問題のチュートリアルと同様な手順で計算して得られる convolution.txt に相当しています。(順問題のチュートリアルとは bulk.exe , surf.exe の入力ファイルが異なるのでご注意ください。)

3.2.3 入力ファイルの説明

メインプログラム用の入力ファイル input.toml の準備をします。input.toml の詳細については入力ファイルに記載されています。ここでは、サンプルファイルにある input.toml の中身について説明します。

```
[base]
dimension = 3
output_dir = "output"

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"
generate_rocking_curve = true
```

(次のページに続く)

(前のページからの続き)

```
[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02", "value_03" ]

[solver.reference]
path = "experiment.txt"
exp_number = [1]

[solver.post]
normalization = "TOTAL"

[algorithm]
name = "minsearch"
label_list = ["z1", "z2", "z3"]

[algorithm.param]
min_list = [0.0, 0.0, 0.0]
max_list = [10.0, 10.0, 10.0]
initial_list = [5.25, 4.25, 3.50]
```

最初に [base] セクションについて説明します。

- dimension は最適化したい変数の個数で、今の場合は template.txt で説明したように 3 つの変数の最適化を行うので、3 を指定します。
- output_dir は出力先のディレクトリ名です。省略した場合はプログラムを実行したディレクトリになります。

[solver] セクションではメインプログラムの内部で使用するソルバーとその設定を指定します。

- name は使用したいソルバーの名前です。 sim-trhepd-rheed に固定されています。
- run_scheme はソルバーを実行する方法の指定です。 subprocess のみ指定可能です。
- generate_rocking_curve は実行ステップごとに Rocking Curve を出力するかどうかを指定します。

ソルバーの設定は、サブセクションの [solver.config], [solver.param], [solver.reference], [solver.post] で行います。

[solver.config] セクションではメインプログラム内部で呼び出す surf.exe により得られた出力ファイルを読み込む際のオプションを指定します。

- cal_number は出力ファイルの何列目を読み込むかを指定します。

[solver.param] セクションではメインプログラム内部で呼び出す surf.exe への入力パラメータについてのオプションを指定します。

- string_list は、 template.txt に記述する、動かしたい変数の名前のリストです。

[solver.reference] セクションでは、実験データの置いてある場所と読みこむ範囲を指定します。

- path は実験データが置いてあるパスを指定します。
- exp_number は実験データファイルの何列目を読み込むかを指定します。

[solver.post] セクションでは、後処理のオプションを指定します。

- normalization は複数ビームの規格化を指定します。

[algorithm] セクションでは、使用するアルゴリスムとその設定をします。

- name は使用したいアルゴリズムの名前で、このチュートリアルでは、Nelder-Mead 法 を用いた解析を行うので、 minsearch を指定します。
- label_list は、value_0x (x=1,2,3) を出力する際につけるラベル名のリストです。

[algorithm.param] セクションでは、探索するパラメータの範囲や初期値を指定します。

- min_list と max_list はそれぞれ探索範囲の最小値と最大値を指定します。
- initial_list は初期値を指定します。

ここではデフォルト値を用いるため省略しましたが、その他のパラメータ、例えば Nelder-Mead 法で使用する収束判定などについては、 [algorithm] セクションで行うことが可能です。詳細については入出力の章をご覧ください。

3.2.4 計算実行

最初にサンプルファイルが置いてあるフォルダへ移動します (以下、本ソフトウェアをダウンロードしたディレクトリ直下にいることを仮定します).

```
$ cd sample/single_beam/minsearch
```

順問題の時と同様に、 bulk.exe と surf.exe をコピーします。

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

bulk.exe を実行し、 bulkP.b を作成します。

```
$ ./bulk.exe
```

そのあとに、メインプログラムを実行します (計算時間は通常の PC で数秒程度で終わります)。

```
$ odatse-STR input.toml | tee log.txt
```

実行すると、以下の様な出力がされます。

```
name            : minsearch
label_list      : ['z1', 'z2', 'z3']
param.min_list  : [0.0, 0.0, 0.0]
param.max_list  : [10.0, 10.0, 10.0]
```

(次のページに続く)

(前のページからの続き)

```
param.initial_list: [5.25, 4.25, 3.5]
eval: x=[5.25 4.25 3.5 ], fun=0.015199252435883206
eval: x=[5.22916667 4.3125     3.64583333], fun=0.013702918645281299
eval: x=[5.22569444 4.40625    3.54513889], fun=0.01263527811899235
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.2185245  4.32627234 3.56743818], fun=0.0032531465329236025
...
```

x= に各ステップでの候補パラメータと、その時の R-factor の値 fun= が出力されます。また各ステップでの計算結果は output/0/LogXXXX_YYYY (XXXX, YYYY はステップ数) のフォルダに出力されます。最終的に推定されたパラメータは、 output/res.dat に出力されます。今の場合、

```
z1 = 5.230573154947887
z2 = 4.3706085364407095
z3 = 3.5961303342247577
```

が得られ、正解のデータ ref.txt と同じ値が得られていることがわかります。なお、一括計算するスクリプトとして do.sh を用意しています。do.sh では output/res.txt と ref.txt の差分も比較しています。以下、説明は割愛しますが、その中身を掲載します。

```
#!/bin/sh

sh ./prepare.sh

./bulk.exe

time odatse-STR input.toml | tee log.txt

echo diff output/res.txt ref.txt
res=0
diff output/res.txt ref.txt || res=$?
if [ $res -eq 0 ]; then
  echo Test PASS
  true
else
  echo Test FAILED: res.txt and ref.txt differ
  false
fi
```

3.2.5 計算結果の可視化

それぞれのステップでのロッキングカーブのデータは、 output/0/LogXXXX_00000001 (XXXX はステップ数) フォルダに RockingCurve.txt として保存されています (output/0/LogXXXX_00000000 フォルダは Nelder-Mead 法の途中での評価です)。このデータを可視化するツール draw_RC_double.py が用意されています。ここでは、このツールを利用して結果を可視化します。

```
$ cp output/0/Log00000000_00000000/RockingCurve_calculated.txt RockingCurve_ini.txt
$ cp output/0/Log00000117_00000000/RockingCurve_calculated.txt RockingCurve_con.txt
$ cp ../../../script/draw_RC_double.py .
$ python3 draw_RC_double.py
```

上記を実行すると RC_double.png が出力されます。

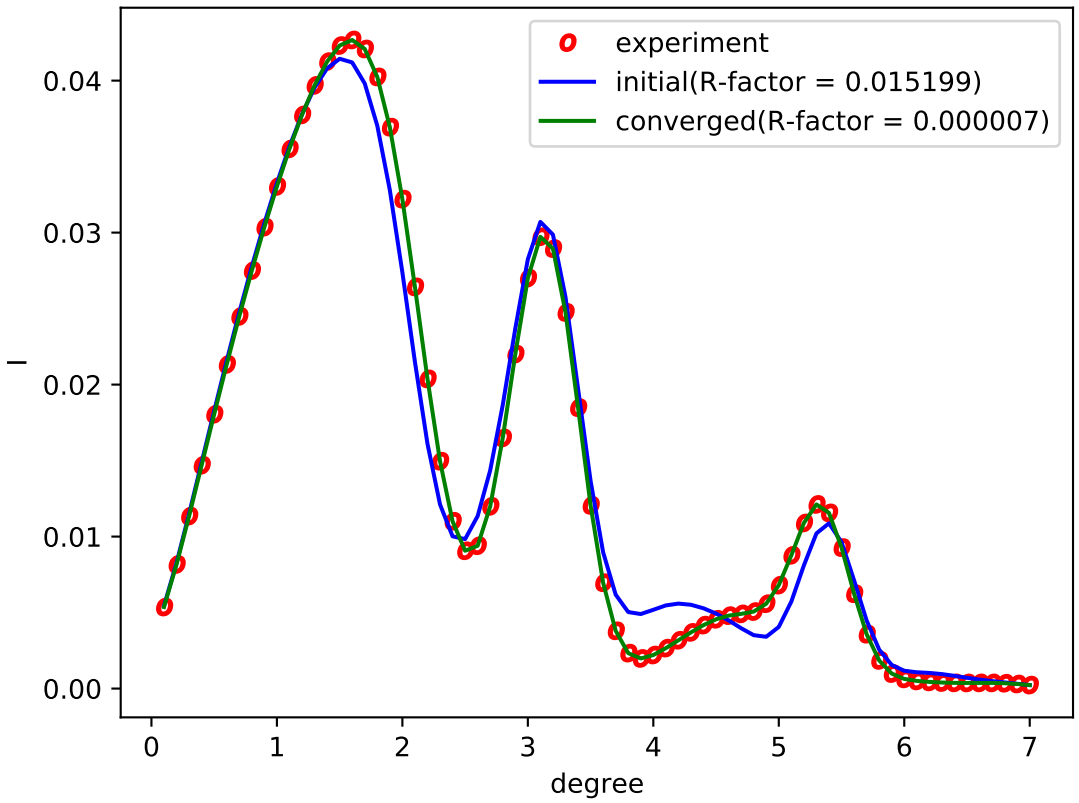


図 3: Nelder-Mead 法を用いた解析。赤丸が実験値、青線が最初のステップ、緑線が最後のステップで得られたロッキングカーブを表す。

図から最後のステップでは実験値と一致していることがわかります。

3.3 グリッド型探索

ここでは、グリッド型探索を行い、回折データから原子座標を解析する方法について説明します。グリッド型探索は MPI に対応しています。具体的な計算手順は minsearch の時と同様です。ただし、探索グリッドを与えるデータ MeshData.txt を事前に準備する必要があります。

3.3.1 サンプルファイルの場所

サンプルファイルは sample/single_beam/mapper にあります。フォルダには以下のファイルが格納されています。

- bulk.txt

  bulk.exe の入力ファイル

- experiment.txt, template.txt

  メインプログラムでの計算を進めるための参照ファイル

- MeshData.txt

  探索グリッドのデータ

- ref_ColorMap.txt

  計算が正しく実行されたか確認するためのファイル (本チュートリアルを行うことで得られる ColorMap.txt の回答)。

- input.toml

  メインプログラムの入力ファイル

- prepare.sh , do.sh

  本チュートリアルを一括計算するために準備されたスクリプト

以下、これらのファイルについて説明したあと、実際の計算結果を紹介します。

3.3.2 参照ファイルの説明

template.txt , experiment.txt については、Nelder-Mead 法による最適化と同じものを使用します。ただし、計算を軽くするため value_03 は用いずに 3.5 に固定し、2 次元のグリッド探索を行うように変更してあります。実際に探索するグリッドは MeshData.txt で与えます。サンプルでは MeshData.txt の中身は以下のようになっています。

```
1 6.000000 6.000000
2 6.000000 5.750000
3 6.000000 5.500000
4 6.000000 5.250000
5 6.000000 5.000000
6 6.000000 4.750000
7 6.000000 4.500000
8 6.000000 4.250000
9 6.000000 4.000000
...
```

1 列目が通し番号、2 列目以降は template.txt に入る value_0, value_1 の値が順に指定されています。

3.3.3 入力ファイルの説明

ここでは、メインプログラム用の入力ファイル input.toml について説明します。input.toml の詳細については入力ファイルに記載されています。以下は、サンプルファイルにある input.toml の内容です。

```
[base]
dimension = 2
output_dir = "output"

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"
generate_rocking_curve = false

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]

[solver.post]
normalization = "TOTAL"

[solver.reference]
path = "experiment.txt"
exp_number = [1]

[algorithm]
name = "mapper"
label_list = ["z1", "z2"]

[algorithm.param]
mesh_path = "./MeshData.txt"
```

最初に [base] セクションについて説明します。

- dimension は最適化したい変数の個数で、今の場合は template.txt で説明したように 2 つの変数の最適化を行うので、2 を指定します。
- output_dir は出力先のディレクトリ名です。省略した場合はプログラムを実行したディレクトリになります。

[solver] セクションではメインプログラムの内部で使用するソルバーとその設定を指定します。

- name は使用したいソルバーの名前です。 sim-trhepd-rheed に固定されています。
- run_scheme はソルバーを実行する方法の指定です。 subprocess のみ指定可能です。

ソルバーの設定は、サブセクションの [solver.config], [solver.param], [solver.reference], [solver.post] で行います。

[solver.config] セクションではメインプログラム内部で呼び出す surf.exe により得られた出力ファイルを読み込む際のオプションを指定します。

- cal_number は出力ファイルの何列目を読み込むかを指定します。

[solver.param] セクションではメインプログラム内部で呼び出す surf.exe への入力パラメータについてのオプションを指定します。

- string_list は、 template.txt で読み込む、動かしたい変数の名前のリストです。

[solver.reference] セクションでは、実験データの置いてある場所と読みこむ範囲を指定します。

- path は実験データが置いてあるパスを指定します。
- exp_number は実験データファイルの何列目を読み込むかを指定します。

[solver.post] セクションでは、後処理のオプションを指定します。

- normalization は複数ビームの規格化を指定します。

[algorithm] セクションでは、使用するアルゴリズムとその設定をします。

- name は使用したいアルゴリズムの名前で、このチュートリアルでは、グリッド探索による解析を行うので、 mapper を指定します。
- label_list は、value_0x (x=1,2) を出力する際につけるラベル名のリストです。

[algorithm.param] セクションでは、探索するパラメータの範囲や初期値を指定します。

- mesh_path は、探索グリッドを記述するファイルを指定します。

その他、入力ファイルで指定可能なパラメータの詳細については入出力の章をご覧ください。

3.3.4 計算実行

最初にサンプルファイルが置いてあるフォルダへ移動します (以下、本ソフトウェアをダウンロードしたディレクトリ直下にいることを仮定します).

```
$ cd sample/single_beam/mapper
```

順問題の時と同様に、 bulk.exe と surf.exe をコピーします。

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

bulk.exe を実行し、 bulkP.b を作成します。

```
$ ./bulk.exe
```

そのあとに、メインプログラムを実行します (計算時間は通常の PC で数秒程度で終わります)。

```
$ mpiexec -np 2 odatse-STR input.toml | tee log.txt
```

ここではプロセス数 2 の MPI 並列を用いた計算を行っています。

実行すると、output ディレクトリ内に各ランクのフォルダが作成され、その中にグリッドの id がついたサブフォルダ LogXXXX_00000000 (XXXX がグリッドの id) が作成されます (MeshData.txt に付けられた番号がグリッドの id として割り振られます)。以下の様な出力が標準出力に書き出されます。

```
name            : mapper
label_list      : ['z1', 'z2']
param.mesh_path : ./MeshData.txt
Iteration : 1/66
Iteration : 2/66
Iteration : 3/66
Iteration : 4/66
Iteration : 5/66
Iteration : 6/66
...
Iteration : 63/66
Iteration : 64/66
Iteration : 65/66
Iteration : 66/66
[0] minimum_value: 1.51992524e-02 at [5.25, 4.25] (mesh 34)
Make ColorMap
complete main process : rank 00000000/00000001
end of run
```

最終的にグリッド上の全ての点で計算された R-factor は output/ColorMap.txt に出力されます。今回の場合は

```
6.000000 6.000000 0.047852
6.000000 5.750000 0.055011
6.000000 5.500000 0.053190
6.000000 5.250000 0.038905
6.000000 5.000000 0.047674
6.000000 4.750000 0.065919
6.000000 4.500000 0.053675
6.000000 4.250000 0.061261
6.000000 4.000000 0.069351
6.000000 3.750000 0.071868
6.000000 3.500000 0.072739
...
```

のように得られます。1 列目、2 列目に value_01, value_02 の値が、3 列目に R-factor が記載されます。

なお、一括計算するスクリプトとして do.sh を用意しています。do.sh では ColorMap.dat と ref_ColorMap.dat の差分も比較しています。以下、説明は割愛しますが、その中身を掲載します。

```
#!/bin/sh
```

(次のページに続く)

(前のページからの続き)

```
sh prepare.sh

./bulk.exe

time mpiexec -np 2 odatse-STR input.toml

echo diff output/ColorMap.txt ref_ColorMap.txt
res=0
diff output/ColorMap.txt ref_ColorMap.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: ColorMap.txt and ref_ColorMap.txt differ
  false
fi
```

3.3.5 計算結果の可視化

ColorMap.txt を図示することで、 R-factor の小さいパラメータがどこにあるかを推定することができます。以下のコマンドを入力すると、2 次元パラメータ空間の図 ColorMapFig.png が作成されます。

```
$ python3 plot_colormap_2d.py
```

作成された図を見ると、(5.25, 4.25) 付近に最小値があることがわかります。

また、 [solver] セクションの generate_rocking_curve パラメータを true にすると、各 Log ディレクトリに RockingCurve_calculated.txt が書き出されます。これを用いることで、前チュートリアルの手順に従い、実験値との比較も行うことが可能です。

3.4 ベイズ最適化

ここでは、ベイズ最適化を行い、回折データから原子座標を解析する方法について説明します。ベイズ最適化には PHYSBO を用いています。グリッド探索と同様に、探索グリッドを与えるデータ MeshData.txt を事前に準備する必要があります。

3.4.1 サンプルファイルの場所

サンプルファイルは sample/single_beam/bayes にあります。フォルダには以下のファイルが格納されています。

- bulk.txt

  bulk.exe の入力ファイル

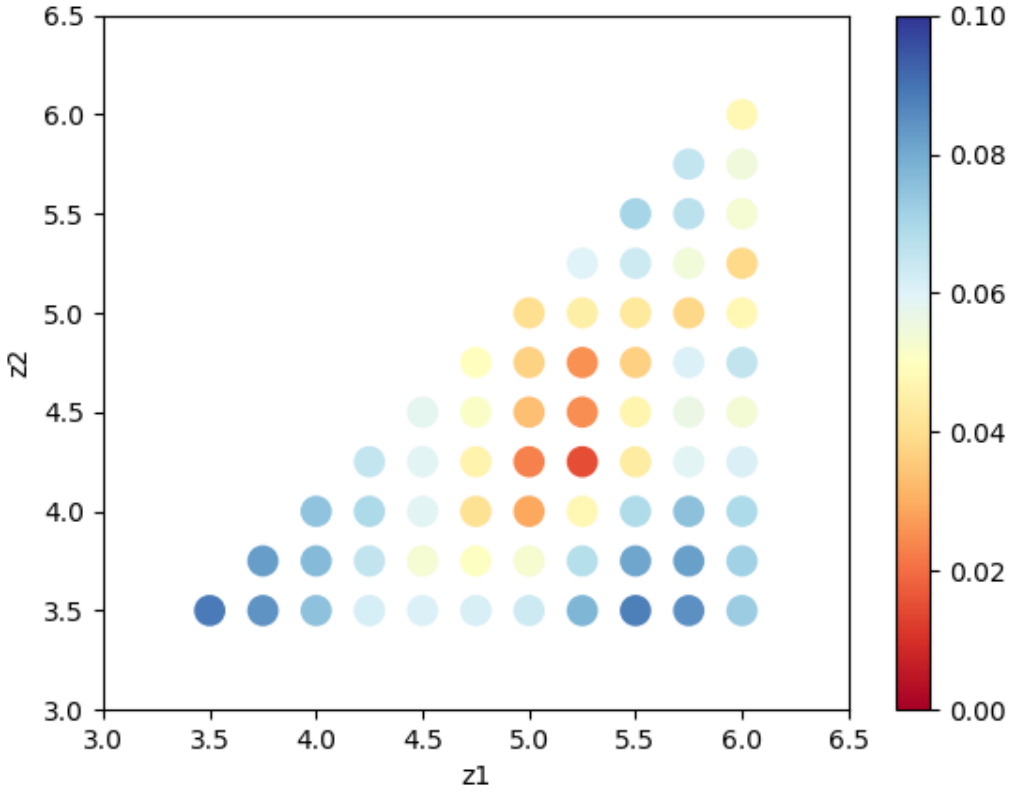


図 4: 2 次元パラメータ空間上での R-factor 。

- experiment.txt, template.txt

  メインプログラムでの計算を進めるための参照ファイル

- MeshData.txt

  探索グリッドのデータ

- ref_BayesData.txt

  計算が正しく実行されたか確認するためのリファレンスファイル

- input.toml

  メインプログラムの入力ファイル

- prepare.sh , do.sh

  本チュートリアルを一括計算するために準備されたスクリプト

以下、これらのファイルについて説明したあと、実際の計算結果を紹介します。

3.4.2 参照ファイルの説明

template.txt, experiment.txt については、Nelder-Mead 法による最適化と同じものを使用します。ただし、計算を軽くするため value_03 は用いずに 3.5 に固定し、2 次元のグリッド探索を行うように変更してあります。実際に探索するグリッドは MeshData.txt で与えます。サンプルでは MeshData.txt の中身は以下のようになっています。

```
1 3.5 3.5
2 3.6 3.5
3 3.6 3.6
4 3.7 3.5
5 3.7 3.6
6 3.7 3.7
7 3.8 3.5
8 3.8 3.6
9 3.8 3.7
10 3.8 3.8
  ...
```

1 列目が通し番号、2 列目以降は template.txt に入る value_0 , value_1 の値が順に指定されています。

3.4.3 入力ファイルの説明

ここでは、メインプログラム用の入力ファイル input.toml について説明します。input.toml の詳細については入力ファイルに記載されています。以下は、サンプルファイルにある input.toml の内容です。

```
[base]
dimension = 2
output_dir = "output"
```

(次のページに続く)

(前のページからの続き)

```
[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"
generate_rocking_curve = false

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]

[solver.reference]
path = "experiment.txt"
exp_number = [1]

[algorithm]
name = "bayes"
label_list = ["z1", "z2"]
seed = 1

[algorithm.param]
mesh_path = "MeshData.txt"

[algorithm.bayes]
random_max_num_probes = 10
bayes_max_num_probes = 20
```

最初に [base] セクションについて説明します。

- dimension は最適化したい変数の個数で、今の場合は template.txt で説明したように 2 つの変数の最適化を行うので、 2 を指定します。
- output_dir は出力先のディレクトリ名です。省略した場合はプログラムを実行したディレクトリになります。

[solver] セクションではメインプログラムの内部で使用するソルバーとその設定を指定します。

- name は使用したいソルバーの名前です。 sim-trhepd-rheed に固定されています。
- run_scheme はソルバーを実行する方法の指定です。 subprocess のみ指定可能です。

ソルバーの設定は、サブセクションの [solver.config], [solver.param], [solver.reference], [solver.post] で行います。

[solver.config] セクションではメインプログラム内部で呼び出す surf.exe により得られた出力ファイルを読み込む際のオプションを指定します。

- cal_number は出力ファイルの何列目を読み込むかを指定します。

[solver.param] セクションではメインプログラム内部で呼び出す surf.exe への入力パラメータについてのオプションを指定します。

- string_list は、 template.txt で読み込む、動かしたい変数の名前のリストです。

[solver.reference] セクションでは、実験データの置いてある場所と読みこむ範囲を指定します。

- path は実験データが置いてあるパスを指定します。
- exp_number は実験データファイルの何列目を読み込むかを指定します。

[solver.post] セクションでは、後処理のオプションを指定します。

- normalization は複数ビームの規格化を指定します。

[algorithm] セクションでは、使用するアルゴリズムとその設定をします。

- name は使用したいアルゴリズムの名前で、このチュートリアルでは、ベイズ最適化による解析を行うので、 bayes を指定します。
- label_list は、 value_0x (x=1,2) を出力する際につけるラベル名のリストです。

[algorithm.param] セクションで、探索パラメータを設定します。

- mesh_path はメッシュファイルへのパスを設定します。

[algorithm.bayes] セクションでは、ベイズ最適化のハイパーパラメータを設定します。

- random_max_num_probes は、ベイズ最適化を行う前のランダム探索する回数を指定します。
- bayes_max_num_probes は、ベイズ探索を行う回数を指定します。

その他、入力ファイルで指定可能なパラメータの詳細については入力ファイルの章をご覧ください。

3.4.4 計算実行

最初にサンプルファイルが置いてあるフォルダへ移動します (以下、本ソフトウェアをダウンロードしたディレクトリ直下にいることを仮定します).

```
$ cd sample/single_beam/bayes
```

順問題の時と同様に、 bulk.exe と surf.exe をコピーします。

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

最初に bulk.exe を実行し、 bulkP.b を作成します。

```
$ ./bulk.exe
```

そのあとに、メインプログラムを実行します (計算時間は通常の PC で数秒程度で終わります)。

```
$ odatse-STR input.toml | tee log.txt
```

実行すると以下の様な標準出力がされます。

```
# parameter
random_max_num_probes = 10
bayes_max_num_probes = 20
score = TS
interval = 5
num_rand_basis = 5000
name            : bayes
label_list      : ['z1', 'z2']
seed            : 1
param.mesh_path : ./MeshData.txt
bayes.random_max_num_probes: 10
bayes.bayes_max_num_probes: 20
0001-th step: f(x) = -0.037237 (action=150)
   current best f(x) = -0.037237 (best action=150)

0002-th step: f(x) = -0.060508 (action=36)
   current best f(x) = -0.037237 (best action=150)

0003-th step: f(x) = -0.062158 (action=175)
   current best f(x) = -0.037237 (best action=150)

0004-th step: f(x) = -0.049211 (action=85)
   current best f(x) = -0.037237 (best action=150)

0005-th step: f(x) = -0.083945 (action=255)
   current best f(x) = -0.037237 (best action=150)

0006-th step: f(x) = -0.055569 (action=170)
   current best f(x) = -0.037237 (best action=150)
...
```

最初に設定したパラメータのリスト、そのあとに各ステップでの候補パラメータと、その時の R-factor に -1 を乗じた f(x) が出力されます。また、その時点での一番良いスコアを持つグリッドインデックス (action) とその場合の f(x) と変数が出力されます。出力ディレクトリ output/0/ の下には更にグリッドの id がついたサブフォルダ LogXXXX_00000000 (XXXX がグリッドの id) が作成され、ソルバーの出力が保存されます。(MeshData.txt に付けられた番号がグリッドの id として割り振られます。) 最終的に推定されたパラメータは output/BayesData.txt に出力されます。

今回の場合は

```
#step z1 z2 fx z1_action z2_action fx_action
0 5.1 4.9 0.037237314010261195 5.1 4.9 0.037237314010261195
1 5.1 4.9 0.037237314010261195 4.3 3.5 0.06050786306685965
2 5.1 4.9 0.037237314010261195 5.3 3.9 0.06215778000834068
3 5.1 4.9 0.037237314010261195 4.7 4.2 0.049210767760634364
4 5.1 4.9 0.037237314010261195 5.7 3.7 0.08394457854191653
```

(次のページに続く)

(前のページからの続き)

```
5 5.1 4.9 0.037237314010261195 5.2 5.2 0.05556857782716691
6 5.1 4.9 0.037237314010261195 5.7 4.0 0.0754639895013157
7 5.1 4.9 0.037237314010261195 6.0 4.4 0.054757310814479355
8 5.1 4.9 0.037237314010261195 6.0 4.2 0.06339787375966344
9 5.1 4.9 0.037237314010261195 5.7 5.2 0.05348404677676544
10 5.1 4.7 0.03002813055356341 5.1 4.7 0.03002813055356341
11 5.1 4.7 0.03002813055356341 5.0 4.4 0.03019977423448576
12 5.3 4.5 0.02887504880071686 5.3 4.5 0.02887504880071686
13 5.1 4.5 0.025865346123665988 5.1 4.5 0.025865346123665988
14 5.2 4.4 0.02031077875240244 5.2 4.4 0.02031077875240244
15 5.2 4.4 0.02031077875240244 5.2 4.6 0.023291891689059388
16 5.2 4.4 0.02031077875240244 5.2 4.5 0.02345999725278686
17 5.2 4.4 0.02031077875240244 5.1 4.4 0.022561543431398066
18 5.2 4.4 0.02031077875240244 5.3 4.4 0.02544527153306051
19 5.2 4.4 0.02031077875240244 5.1 4.6 0.02778877135528466
20 5.2 4.3 0.012576357659158034 5.2 4.3 0.012576357659158034
21 5.1 4.2 0.010217361468113488 5.1 4.2 0.010217361468113488
22 5.1 4.2 0.010217361468113488 5.2 4.2 0.013178053637167673
  ...
```

のように得られます。1 列目にステップ数、2 列目、3 列目、4 列目にその時点での最高スコアを与える value_01, value_02 と R-factor が記載されます。続けて、そのステップで候補となった value_01, value_02 と R-factor が記載されます。今回の場合は 21 ステップ目で正しい解が得られていることがわかります。

なお、一括計算するスクリプトとして do.sh を用意しています。do.sh では BayesData.dat と ref_BayesData.dat の差分も比較しています。以下、説明は割愛しますが、その中身を掲載します。

```
#!/bin/sh

sh prepare.sh

./bulk.exe

time odatse-STR input.toml

echo diff output/BayesData.txt ref_BayesData.txt
res=0
diff output/BayesData.txt ref_BayesData.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: BayesData.txt.txt and ref_BayesData.txt.txt differ
  false
fi
```

3.4.5 計算結果の可視化

BayesData.txt を参照することで、何ステップ目のパラメータが最小スコアを与えたかがわかります。[solver] セクションの generate_rocking_curve パラメータを true にすると、各ステップ毎のサブフォルダに RockingCurve_calculated.txt が格納されます。Nelder-Mead 法による最適化での手順に従い、実験値との比較を行うことが可能です。

3.5 レプリカ交換モンテカルロ法による探索

ここでは、レプリカ交換モンテカルロ法によって、回折データから原子座標を解析する方法について説明します。具体的な計算手順は Nelder-Mead 法による最適化と同様です。

3.5.1 サンプルファイルの場所

サンプルファイルは sample/single_beam/exchange にあります。フォルダには以下のファイルが格納されています。

- bulk.txt

  bulk.exe の入力ファイル

- experiment.txt , template.txt

  メインプログラムでの計算を進めるための参照ファイル

- ref.txt

  計算が正しく実行されたか確認するためのファイル (本チュートリアルを行うことで得られる best_result.txt の回答)。

- input.toml

  メインプログラムの入力ファイル

- prepare.sh , do.sh

  本チュートリアルを一括計算するために準備されたスクリプト

以下、これらのファイルについて説明したあと、実際の計算結果を紹介します。

3.5.2 参照ファイルの説明

template.txt, experiment.txt については、Nelder-Mead 法による最適化と同じものを使用します。

3.5.3 入力ファイルの説明

ここでは、メインプログラム用の入力ファイル input.toml について説明します。input.toml の詳細については入力ファイルに記載されています。以下は、サンプルファイルにある input.toml の内容です。

```
[base]
dimension = 2
output_dir = "output"

[algorithm]
name = "exchange"
label_list = ["z1", "z2"]
seed = 12345

[algorithm.param]
min_list = [3.0, 3.0]
max_list = [6.0, 6.0]
step_list = [1.0, 1.0]

[algorithm.exchange]
numsteps = 1000
numsteps_exchange = 20
Tmin = 0.005
Tmax = 0.05
Tlogspace = true

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]
degree_max = 7.0

[solver.reference]
path = "experiment.txt"
exp_list = [1]

[solver.post]
normalization = "TOTAL"
```

ここではこの入力ファイルを簡単に説明します。詳細は入力ファイルのレファレンスを参照してください。

[base] セクションはメインプログラム全体のパラメータです。dimension は最適化したい変数の個数で、今の場合は 2 つの変数の最適化を行うので、2 を指定します。output_dir は出力先のディレクトリ名です。省略した場合はプログラムを実行したディレクトリになります。

[algorithm] セクションは用いる探索アルゴリズムを設定します。交換モンテカルロ法を用いる場合には、name に "exchange" を指定します。label_list は、value_0x (x=1,2) を出力する際につけるラベル名の

リストです。seed は擬似乱数生成器に与える種です。

[algorithm.param] サブセクションは、最適化したいパラメータの範囲などを指定します。min_list は最小値、 max_list は最大値を示します。 step_list はモンテカルロ更新の際の変化幅 (ガウス分布の偏差) です。

[algorithm.exchange] サブセクションは、交換モンテカルロ法のハイパーパラメータを指定します。

- numstep はモンテカルロ更新の回数です。
- numsteps_exchange で指定した回数のモンテカルロ更新の後に、温度交換を試みます。
- Tmin, Tmax はそれぞれ温度の下限・上限です。
- Tlogspace が true の場合、温度を対数空間で等分割します

[solver] セクションではメインプログラムの内部で使用するソルバーを指定します。Nelder-Mead 法による最適化のチュートリアルを参照してください。

3.5.4 計算実行

最初にサンプルファイルが置いてあるフォルダへ移動します (以下、本ソフトウェアをダウンロードしたディレクトリ直下にいることを仮定します).

```
$ cd sample/single_beam/exchange
```

順問題の時と同様に、 bulk.exe と surf.exe をコピーします。

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

最初に bulk.exe を実行し、 bulkP.b を作成します。

```
$ ./bulk.exe
```

そのあとに、メインプログラムを実行します (計算時間は通常の PC で数秒程度で終わります)。

```
$ mpiexec -np 4 odatse-STR input.toml | tee log.txt
```

ここではプロセス数 4 の MPI 並列を用いた計算を行っています。(Open MPI を用いる場合で、使えるコア数よりも要求プロセス数の方が多い時には、mpiexec コマンドに --oversubscribe オプションを追加してください。)

実行すると、 output ディレクトリ内に各ランクのフォルダが作成され、各モンテカルロステップで評価したパラメータおよび目的関数の値を記した trial.txt ファイルと、実際に採択されたパラメータを記した result.txt ファイルが作成されます。ともに書式は同じで、最初の 2 列がステップ数とプロセス内の walker 番号、次が温度、3 列目が目的関数の値、4 列目以降がパラメータです。

```
# step walker T fx x1 x2
0 0 0.004999999999999999 0.07830821484593968 3.682008067401509 3.9502750191292586
1 0 0.004999999999999999 0.07830821484593968 3.682008067401509 3.9502750191292586
```

(次のページに続く)

(前のページからの続き)

```
2 0 0.004999999999999999 0.07830821484593968 3.682008067401509 3.9502750191292586
3 0 0.004999999999999999 0.06273922648753057 4.330900869594549 4.311333132184154
```

また、各作業フォルダの下にサブフォルダ LogXXXX_00000000 (XXXX がグリッドの id) が作成され、ロッキングカーブの情報などが記録されます。(各プロセスにおけるモンテカルロステップ数が id として割り振られます。)

各ランクフォルダにある result.txt には、各レプリカでサンプリングされたデータが記録されていますが、これを温度ごとのサンプルに整列しなおしたデータが output/result_T*.txt として出力されます。* は温度のインデックスを表します。

最後に、 output/best_result.txt に、目的関数 (R-factor) が最小となったパラメータとそれを得たランク、モンテカルロステップの情報が書き込まれます。

```
nprocs = 4
rank = 1
step = 282
walker = 0
fx = 0.008414800224430936
z1 = 5.164773671165013
z2 = 4.226467514644945
```

なお、一括計算するスクリプトとして do.sh を用意しています。do.sh では best_result.txt と ref.txt の差分も比較しています。以下、説明は割愛しますが、その中身を掲載します。

```
#!/bin/sh

sh prepare.sh

./bulk.exe

time mpiexec --oversubscribe -np 4 odatse-STR input.toml

echo diff output/best_result.txt ref.txt
res=0
diff output/best_result.txt ref.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: best_result.txt and ref.txt differ
  false
fi
```

3.5.5 計算結果の可視化

output/result_T*.txt を図示することで、R-factor の小さいパラメータがどこにあるかを推定することができます。以下のコマンドを実行すると、output/result_T1.txt のサンプル点をプロットした 2 次元パラメータ空間の図 result.png が作成されます。

```
$ python3 plot_result_2d.py
```

作成された図を見ると、(5.25, 4.25) と (4.25, 5.25) 付近にサンプルが集中していることと、R-factor の値が小さいことがわかります。

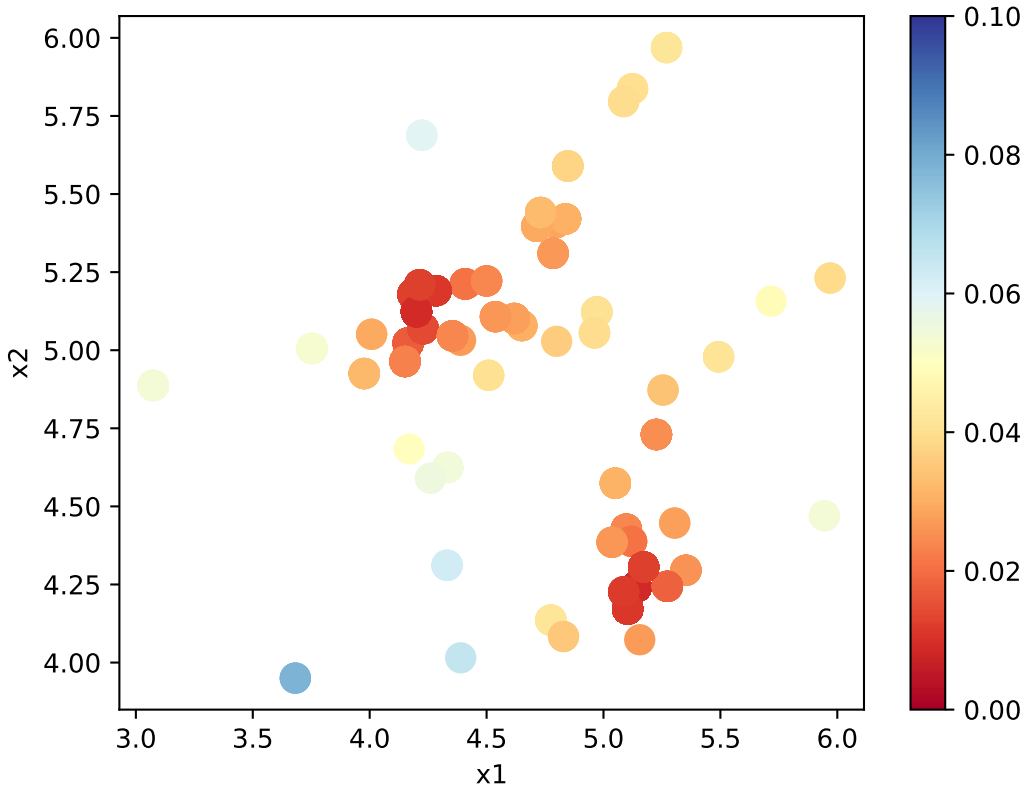


図 5: サンプルされたパラメータと R-factor 。横軸は value_01, 縦軸は value_02 を表す。

また、 [solver] セクションの generate_rocking_curve パラメータを true にすると、RockingCurve_calculated.txt が各サブフォルダに格納されます。これを用いることで、Nelder-Mead 法による最適化での手順に従い、実験値との比較も行うことが可能です。

3.6 ポピュレーションアニーリングによる探索

ここでは、ポピュレーションアニーリングを用いて、回折データから原子座標を解析する方法について説明します。具体的な計算手順は Nelder-Mead 法による最適化と同様です。

3.6.1 サンプルファイルの場所

サンプルファイルは sample/single_beam/pamc にあります。フォルダには以下のファイルが格納されています。

- bulk.txt

  bulk.exe の入力ファイル

- experiment.txt, template.txt

  メインプログラムでの計算を進めるための参照ファイル

- ref.txt

  計算が正しく実行されたか確認するためのファイル (本チュートリアルを行うことで得られる fx.txt の回答)。

- input.toml

  メインプログラムの入力ファイル

- prepare.sh , do.sh

  本チュートリアルを一括計算するために準備されたスクリプト

以下、これらのファイルについて説明したあと、実際の計算結果を紹介します。

3.6.2 参照ファイルの説明

template.txt, experiment.txt については、Nelder-Mead 法による最適化と同じものを使用します。

3.6.3 入力ファイルの説明

ここでは、メインプログラム用の入力ファイル input.toml について説明します。input.toml の詳細については入力ファイルに記載されています。以下は、サンプルファイルにある input.toml の内容です。

```
[base]
dimension = 2
output_dir = "output"

[algorithm]
name = "pamc"
label_list = ["z1", "z2"]
seed = 12345

[algorithm.param]
min_list = [3.0, 3.0]
max_list = [6.0, 6.0]
step_list = [0.3, 0.3]

[algorithm.pamc]
```

(次のページに続く)

(前のページからの続き)

```
numsteps_annealing = 5
bmin = 0.0
bmax = 200.0
Tnum = 21
Tlogspace = false
nreplica_per_proc = 10

[solver]
name = "sim-trhepd-rheed"
run_scheme = "subprocess"

[solver.config]
cal_number = [1]

[solver.param]
string_list = ["value_01", "value_02" ]

[solver.post]
normalization = "TOTAL"

[solver.reference]
path = "experiment.txt"
exp_number = [1]
```

ここではこの入力ファイルを簡単に説明します。詳細は入力ファイルのレファレンスを参照してください。

[base] セクションはメインプログラム全体のパラメータです。dimension は最適化したい変数の個数で、今の場合は 2 つの変数の最適化を行うので、2 を指定します。output_dir は出力先のディレクトリ名です。省略した場合はプログラムを実行したディレクトリになります。

[algorithm] セクションは用いる探索アルゴリズムを設定します。交換モンテカルロ法を用いる場合には、name に "exchange" を指定します。label_list は、value_0x (x=1,2) を出力する際につけるラベル名のリストです。seed は擬似乱数生成器に与える種です。

[algorithm.param] サブセクションは、最適化したいパラメータの範囲などを指定します。min_list は最小値、 max_list は最大値を示します。unit_list はモンテカルロ更新の際の変化幅 (ガウス分布の偏差) です。

[algorithm.pamc] サブセクションは、ポピュレーションアニーリングのハイパーパラメータを指定します。

- numsteps_annealing で指定した回数のモンテカルロ更新の後に、逆温度を増やします (温度を下げます)。
- bmin, bmax はそれぞれ逆温度の下限・上限です。
- Tnum は計算する温度・逆温度の点数です。
- Tlogspace が true の場合、温度を対数空間で等分割します
- nreplica_per_proc は MPI プロセスひとつが受け持つ計算レプリカの数です。

[solver] セクションではメインプログラムの内部で使用するソルバーを指定します。Nelder-Mead 法による最適化のチュートリアルを参照してください。

3.6.4 計算実行

最初にサンプルファイルが置いてあるフォルダへ移動します (以下、本ソフトウェアをダウンロードしたディレクトリ直下にいることを仮定します).

```
$ cd sample/single_beam/pamc
```

順問題の時と同様に、 bulk.exe と surf.exe をコピーします。

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

最初に bulk.exe を実行し、 bulkP.b を作成します。

```
$ ./bulk.exe
```

そのあとに、メインプログラムを実行します (計算時間は通常の PC で数秒程度で終わります)。

```
mpiexec -np 4 odatse-STR input.toml | tee log.txt
```

ここではプロセス数 4 の MPI 並列を用いた計算を行っています。(Open MPI を用いる場合で、使えるコア数よりも要求プロセス数の方が多い時には、 mpiexec コマンドに --oversubscribe オプションを追加してください。)

実行すると、 output ディレクトリの中に各ランクのフォルダが作成され、温度ごとに各モンテカルロステップで評価したパラメータおよび目的関数の値を記した trial_TXXX.txt ファイル (XXX は温度点の番号) と、実際に採択されたパラメータを記した result_TXXX.txt ファイル、さらにそれぞれを結合した trial.txt, result.txt ファイルが生成されます。それぞれ書式は同じで、最初の 2 列がステップ数とプロセス内の walker (replica) 番号、次が (逆) 温度、3 列目が目的関数の値、4 列目以降がパラメータです。最後の 2 列は、 walker の重み (Neal-Jarzynski weight) と祖先 (計算を開始したときのレプリカ) の番号です。

```
# step walker beta fx z1 z2 weight ancestor
0 0 0.0 0.07702743614780189 5.788848278451443 3.949126663745358 1.0 0
0 1 0.0 0.08737730661436376 3.551756435031283 3.6136808356591192 1.0 1
0 2 0.0 0.04954470587051104 4.70317508724506 4.786634108937754 1.0 2
0 3 0.0 0.04671675601156148 5.893543559206865 4.959531290614713 1.0 3
0 4 0.0 0.04142014655238446 5.246719912601735 4.960709612555206 1.0 4
```

また、各作業フォルダの下にサブフォルダ LogXXXX_00000000 (XXXX がグリッドの id) が作成され、ロッキングカーブの情報などが記録されます (各プロセスにおけるモンテカルロステップ数が id として割り振られます)。

output/best_result.txt に、目的関数 (R-factor) が最小となったパラメータとそれを得たランク、モンテカルロステップの情報が書き込まれます。

```
nprocs = 4
rank = 0
step = 71
walker = 5
fx = 0.008186713312593607
z1 = 4.225633749839847
z2 = 5.142666117413409
```

最後に、 output/fx.txt には、各温度ごとの統計情報が記録されます。

```
# $1: 1/T
# $2: mean of f(x)
# $3: standard error of f(x)
# $4: number of replicas
# $5: log(Z/Z0)
# $6: acceptance ratio
0.0 0.06428002079611472 0.002703413400677839 40 0.0 0.795
10.0 0.061399304916174735 0.002649424392996749 40 -0.6280819199879947 0.85
20.0 0.05904248889111052 0.003162271121295203 4 40 -1.2283060742855603 0.74
30.0 0.04956921148431115 0.002829856575915963 3 40 -1.7991035905899855 0.67
```

1 列目は温度・逆温度で、2・3 列目は目的関数 $f(x)$ の期待値と標準誤差、4 列目はレプリカの個数、5 列目は分配関数の比の対数 $\log(Z_n/Z_0)$ です (Z_0 は最初の温度点における分配関数)、6 列目はモンテカルロ更新の採択率です。

なお、一括計算するスクリプトとして do.sh を用意しています。do.sh では res.txt と ref.txt の差分も比較しています。以下、説明は割愛しますが、その中身を掲載します。

```
#!/bin/sh

sh prepare.sh

./bulk.exe

time mpiexec --oversubscribe -np 4 odatse-STR input.toml

echo diff output/fx.txt ref.txt
res=0
diff output/fx.txt ref.txt || res=$?
if [ $res -eq 0 ]; then
  echo TEST PASS
  true
else
  echo TEST FAILED: output/fx.txt and ref.txt differ
  false
fi
```

3.6.5 計算結果の可視化

output/*/result_T%.txt を図示することで、R-factor の小さいパラメータがどこにあるかを推定することができます。今回の場合は、以下のコマンドを実行すると 2 次元パラメータ空間の図 result_fx.png と result_T.png が作成されます。シンボルの色はそれぞれ R-factor と逆温度 β に対応します。

```
$ python3 plot_result_2d.py
```

作成された図を見ると、(5.25, 4.25) と (4.25, 5.25) 付近にサンプルが集中していることと、R-factor の値が小さいことがわかります。

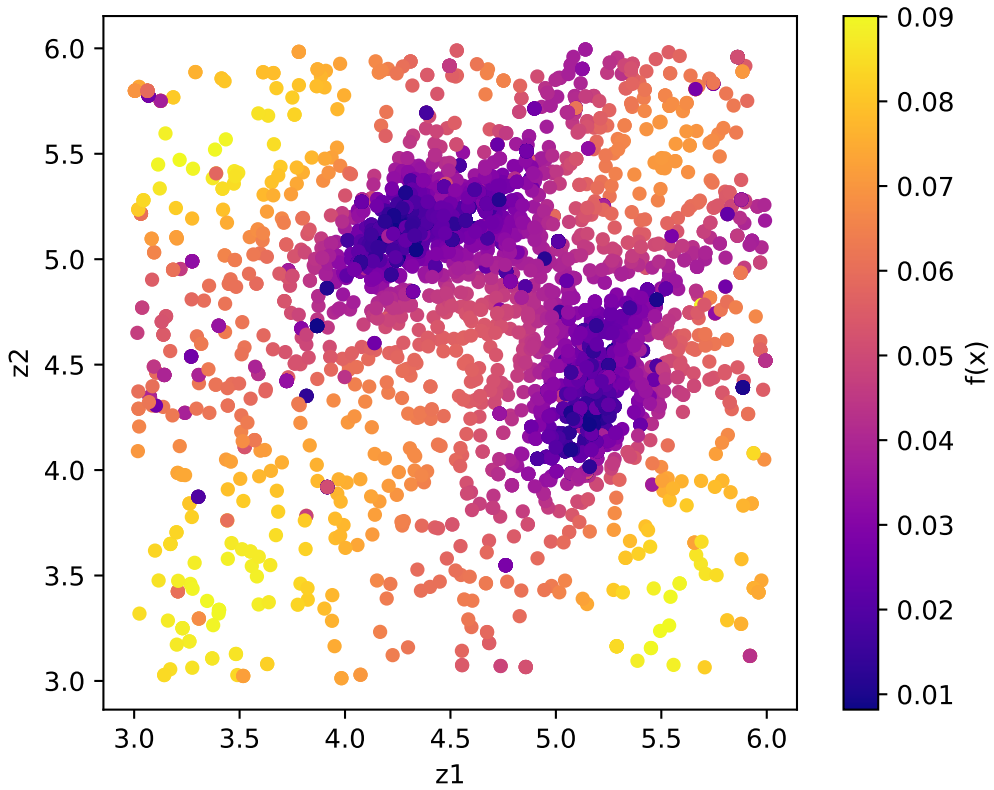


図 6: サンプルされたパラメータ。横軸は value_01 , 縦軸は value_02 を、色は R-factor を表す。

また、 [solver] セクションの generate_rocking_curve パラメータを true にすると、RockingCurve_calculated.txt が各サブフォルダに格納されます。これを用いることで、Nelder-Mead 法による最適化での手順に従い、実験値との比較も行うことが可能です。

3.7 ユーザープログラムによる解析

ここでは、odatse-STR モジュールを用いたユーザプログラムを作成し、解析を行う方法を説明します。逆問題アルゴリズムは例として Nelder-Mead 法を用います。

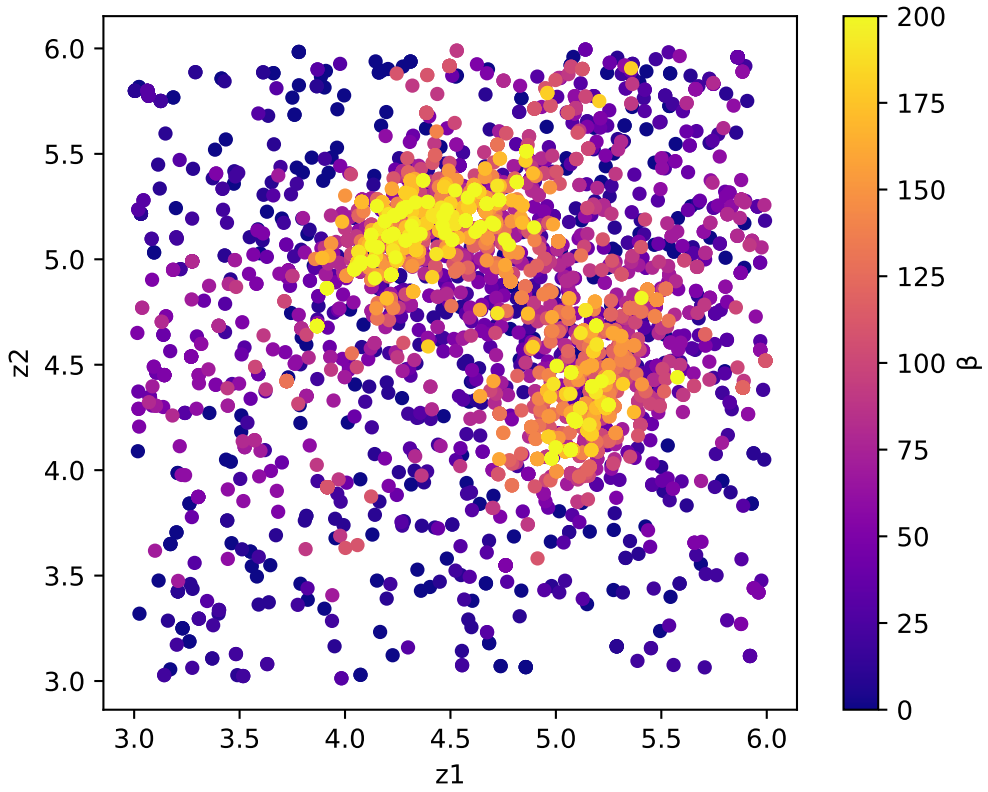


図 7: サンプルされたパラメータと逆温度。横軸は `value_01` , 縦軸は `value_02` を、色は逆温度を表す。

3.7.1 サンプルファイルの場所

サンプルファイルは sample/single_beam/user_program にあります。フォルダには以下のファイルが格納されています。

- simple.py

  メインプログラム。パラメータを input.toml ファイルから読み込んで解析を行う。

- input.toml

  simple.py で利用する入力パラメータファイル

- experiment.txt, template.txt

  メインプログラムでの計算を進めるための参照ファイル

- ref.txt

  本チュートリアルで求めたい回答が記載されたファイル

- simple2.py

  メインプログラムの別バージョン。パラメータを dict 形式でスクリプト内に埋め込んでいる。

- prepare.sh , do.sh

  本チュートリアルを一括計算するために準備されたスクリプト

以下、これらのファイルについて説明したのち、実際の計算結果を紹介します。

3.7.2 プログラムの説明

simple.py は odatse-STR モジュールを用いて解析を行うシンプルなプログラムです。プログラムの全体を以下に示します。

```
import numpy as np

import odatse
import odatse.algorithm.min_search
from odatse.extra.STR import Solver

info = odatse.Info.from_file("input.toml")

odatse.mpi.setup()
solver = Solver(info)
runner = odatse.Runner(solver, info)
alg = odatse.algorithm.min_search.Algorithm(info, runner)
alg.main()
```

プログラムではまず、必要なモジュールを import します。

- ODAT-SE のメインモジュール odatse
- 今回利用する逆問題解析アルゴリズム odatse.algorithm.min_search

- 順問題ソルバーモジュール odatse.extra.STR

次に、解析で利用するクラスのインスタンスを作成します。

- odatse.Info クラス

  パラメータを格納するクラスです。 from_file クラスメソッドに TOML ファイルのパスを渡して作成することができます。

Solver クラスのインスタンスを作成する前に odatse.mpi.setup() を呼び出して MPI の設定を行います。ODAT-SE v4 ではこの呼び出しが必須で、MPI を使わない場合にも必要です。(odatse-STR コマンドで用いられる odatse.initialize() はコマンドライン引数の処理と合わせてこの設定を内部で行います。)

- odatse.extra.STR.Solver クラス

  odatse-STR モジュールの順問題ソルバーです。Info クラスのインスタンスを渡して作成します。

- odatse.Runner クラス

  順問題ソルバーと逆問題解析アルゴリズムを繋ぐクラスです。Solver クラスのインスタンスおよび Info クラスのパラメータを渡して作成します。

- odatse.algorithm.min_search.Algorithm クラス

  逆問題解析アルゴリズムのクラスです。ここでは Nelder-Mead 法による最適化アルゴリズムのクラスモジュール min_search を利用します。Runner のインスタンスを渡して作成します。

Solver, Runner, Algorithm の順にインスタンスを作成した後、Algorithm クラスの main() メソッドを呼んで解析を行います。

上記のプログラムでは、入力パラメータを TOML 形式のファイルから読み込む形ですが、パラメータを dict 形式で渡すこともできます。simple2.py はパラメータをプログラム中に埋め込む形式で記述したものです。以下にプログラムの全体を記載します。

```
import numpy as np

import odatse
import odatse.algorithm.min_search
from odatse.extra.STR import Solver

params = {
    "base": {
        "dimension": 3,
        "output_dir": "output",
    },
    "solver": {
        "run_scheme": "subprocess",
        "generate_rocking_curve": True,
        "config": {
            "cal_number": [1],
        },
        "param": {
```

(次のページに続く)

(前のページからの続き)

```
            "string_list": ["value_01", "value_02", "value_03"],
        },
        "reference": {
            "path": "experiment.txt",
            "exp_number": [1],
        },
        "post": {
            "normalization": "TOTAL",
        },
    },
    "algorithm": {
        "label_list": ["z1", "z2", "z3"],
        "param": {
            "min_list": [ 0.0, 0.0, 0.0 ],
            "max_list": [ 10.0, 10.0, 10.0 ],
            "initial_list": [ 5.25, 4.25, 3.50],
        },
    },
}

info = odatse.Info(params)

odatse.mpi.setup()
solver = Solver(info)
runner = odatse.Runner(solver, info)
alg = odatse.algorithm.min_search.Algorithm(info, runner)
alg.main()
```

dict 形式のパラメータを渡して Info クラスのインスタンスを作成します。同様に、パラメータをプログラム内で生成して渡すこともできます。

3.7.3 入力ファイルの説明

メインプログラム用の入力ファイル `input.toml` は前述の Nelder-Mead 法による最適化で用いたのと同じファイルを利用できます。なお、アルゴリズムの種類を指定する `algorithm.name` パラメータの値は無視されます。

その他、 `template.txt`, `experiment.txt` ファイルは前述の tutorial と同様です。

3.7.4 計算実行

最初にサンプルファイルが置いてあるフォルダへ移動します (以下、本ソフトウェアをダウンロードしたディレクトリ直下にいることを仮定します).

```
$ cd sample/single_beam/user_program
```

bulk.exe と surf.exe をコピーします。

```
$ cp ../../sim-trhepd-rheed/src/bulk.exe .
$ cp ../../sim-trhepd-rheed/src/surf.exe .
```

bulk.exe を実行し、 bulkP.b を作成します。

```
$ ./bulk.exe
```

そのあとに、メインプログラムを実行します (計算時間は通常の PC で数秒程度で終わります)。

```
$ python3 simple.py | tee log.txt
```

実行すると、以下の様な出力がされます。

```
name            : minsearch
label_list      : ['z1', 'z2', 'z3']
param.min_list  : [0.0, 0.0, 0.0]
param.max_list  : [10.0, 10.0, 10.0]
param.initial_list: [5.25, 4.25, 3.5]
eval: x=[5.25 4.25 3.5 ], fun=0.015199252435883206
eval: x=[5.22916667 4.3125     3.64583333], fun=0.013702918645281299
eval: x=[5.22569444 4.40625    3.54513889], fun=0.01263527811899235
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.17997685 4.34895833 3.5943287 ], fun=0.006001659635528168
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.22066294 4.33260995 3.60408629], fun=0.005145496928704404
eval: x=[5.2185245  4.32627234 3.56743818], fun=0.0032531465329236025
eval: x=[5.19953437 4.34549482 3.5863457 ], fun=0.0027579225484420356
eval: x=[5.21549776 4.35100199 3.57476018], fun=0.002464573036316852
...
```

x= に各ステップでの候補パラメータ、その時の R-factor の値が fun= に出力されます。また各ステップでの計算結果は output/0/LogXXXX_YYYY (XXXX, YYYY はステップ数) のフォルダに出力されます。最終的に推定されたパラメータは、 output/res.dat に出力されます。今の場合、

```
z1 = 5.230524973874179
z2 = 4.370622919269477
z3 = 3.5961444501081647
```

となります。リファレンス ref.txt が再現されていることが分かります。

なお、一連の計算を行う do.sh スクリプトが用意されています。

第4章　入出力

odatse-STR モジュールは sim-trhepd-rheed を用いて原子位置 x から回折 rocking curve を計算し、実験で得られた rocking curve からの誤差を $f(x)$ として返す Solver です。

> **注釈**
>
> sim-trhepd-rheed は逐次実行のプログラムです。ODAT-SE v4 のソルバー並列化 (--nsolve オプションに 2 以上を指定) には対応しておらず、指定した場合はエラーで終了します。MPI 並列はアルゴリズム側の並列化 (--nalg、既定では全プロセス) を利用してください。

この章では、入力パラメータおよび入力データと出力データについて説明します。入力パラメータは Info クラスの solver の項目が該当します。TOML ファイルを入力として与える場合は、 [solver] セクションに記述します。dict 形式でパラメータを作成する場合は solver キー以下に入れ子の dict 形式でデータを用意します。以下では、TOML 形式でパラメータ項目を説明します。

入力データは、ターゲットとする参照データと、sim-trhepd-rheed の入力ファイルのテンプレートです。出力データは、 sim-trhepd-rheed の surf.exe が出力する結果およびログファイルです。以下の節で内容を示します。

4.1 入力パラメータ

solver セクションとセクション中のサブセクション config, post, param, reference を利用します。

4.1.1 [solver] セクション

- generate_rocking_curve

  形式: 真偽値 (default: false)

  説明: RockingCurve_calculated.txt の生成をするかどうか。 RockingCurve_calculated.txt は作業ディレクトリ Log%%%_### の中に保存されます。このオプションが"true"であっても、remove_work_dir （[post] セクション）が"true"であれば Log%%%_### は削除されてしまうため、注意してください。

4.1.2 [solver.config] セクション

- surface_exec_file

  形式: string 型 (default: "surf.exe")

  説明: sim-trhepd-rheed の表面反射ソルバー surf.exe へのパス

- surface_input_file

  形式: string 型 (default: "surf.txt")

  説明: 表面構造のインプットファイル。

- bulk_output_file

  形式: string 型 (default: "bulkP.b")

  説明: バルク構造のアウトプットファイル。

- surface_output_file

  形式: string 型 (default: "surf-bulkP.s")

  説明: 表面構造のアウトプットファイル。

- calculated_first_line

  形式: 整数型 (default: 5)

  説明: surface_output_file ファイル中, D(x) として読み込む最初の行。 なお、最終行は実験データとして読み込んだ行数から自動計算されます。

- calculated_info_line

  形式: 整数型 (default: 2)

  説明: surface_output_file ファイル中, glancing angle 数や beam 数が記してある行番号。

- cal_number

  形式: 整数型のリスト (設定必須)

  説明: surface_output_file ファイル中, データとして読み出す列番号。複数指定が可能。設定した値は、後述の exp_number （[reference] セクション）の値とリストの長さを一致させる必要があります。

4.1.3 [solver.post] セクション

実験と計算の「ズレ」の大きさを示し、最小化すべき関数である「目的関数」の定義や、ロッキングカーブを描くためのコンボリューション半値幅などを指定するセクションです。

- Rfactor_type

  形式: string 型。"A"、"A2"または"B"のいずれかをとります。 (default: "A")

  説明: 目的関数値の計算方法の指定。視写角のインデックスを $i = 1, 2, \ldots, m$, ビームのインデックスを $j = 1, 2, \ldots, n$ として、実験・計算のデータをそれぞれ $u_i^{(j)}$ と $v_i^{(j)}$ で表し、各ビームの重みを $w^{(j)}$ と書くとき

  - "A"タイプ:

$$R = \sqrt{\sum_j^n w^{(j)} \sum_i^m \left(u_i^{(j)} - v_i^{(j)} \right)^2}$$

- "A2"タイプ:

$$R^2 = \sum_j^n w^{(j)} \sum_i^m \left(u_i^{(j)} - v_i^{(j)} \right)^2$$

- "B"タイプ:

$$R = \frac{\sum_i^m \left(u_i^{(1)} - v_i^{(1)} \right)^2}{\sum_i^m \left(u_i^{(1)} \right)^2 + \sum_i^m (v_i^{(1)})^2}$$

  * "B"タイプはデータ列が 1 つの実験・計算データを用いた実行 ($n = 1$) のみ対応しています。

- `normalization`

  形式: string 型。"TOTAL"または"MANY_BEAM"のいずれかをとります。 (設定必須)

  説明: 実験・計算のデータベクトルの規格化の方法。

  - "TOTAL"
    * 全体の和で規格化をします。
    * 計算に使用するデータ列が 1 つのみ適用できる規格化方法であり、`cal_number`（[config] セクション）および `exp_number`（[reference] セクション）で設定したリストの長さが 1 である必要が有ります。
  - "MANY_BEAM"
    * "MANY_BEAM"はデータ列が 2 つ以上であるときに利用できる規格化方法です。後述の `weight_type` によって規格化方法が変わります。

  なお、 normalization="MAX" は廃止となりました。

- `weight_type`

  形式: string 型または `None` 。"calc"または"manual"のいずれかを設定する必要があります。 (default: `None` 、 `normalization = "MANY_BEAM"` としたとき設定必須)

  説明: 目的関数値を計算するときの、ビームごとの重み $w^{(j)}$ の計算方法を指定します。"calc"とした場合、データ列ごとの重み $w^{(n)}$ は次の式で与えられます。

$$w^{(j)} = \left(\frac{\sum_{i=1}^m v_i^{(j)}}{\sum_{k=1}^n \sum_{i=1}^m v_i^{(j)}} \right)^2$$

  "manual"とした場合、オプション `spot_weight` を用いることで、ユーザーが重みを指定可能です。

- `spot_weight`

  形式: float 型のリスト。 (default: `[]`, `weight_type = "manual"` としたとき設定必須)

  説明: 目的関数値を計算するときの、データ列ごとの重みを設定します。総和が 1 になるように自動的に規格化されます。例えば、[3,2,1] を指定すると、$w^{(1)} = 1/2, w^{(2)} = 1/3, w^{(3)} = 1/6$ となります。

- `omega`

  形式: 実数型 (default: 0.5)

  説明: コンボリューションの半値幅の指定。

- remove_work_dir

  形式: 真偽値 (default: false)

  説明: R-factor を読み取った後に作業ディレクトリ Log%%%_### を削除するかどうか。なお、generate_rocking_curve ([solver] セクション) が"true"であっても、本オプションが"true"ならば Log%%%_### を削除します。

4.1.4 [solver.param] セクション

- string_list

  形式: string 型のリスト。長さは dimension の値と一致させます。 (default: ["value_01", "value_02"])

  説明: ソルバーの入力ファイルを作成するための参照用テンプレートファイルで利用するプレースホルダーのリスト。これらの文字列が探索中のパラメータの値に置換されます。

4.1.5 [solver.reference] セクション

- path

  形式: string 型 (default: experiment.txt)

  説明: 実験データファイルへのパス。

- reference_first_line

  形式: 整数型

  説明: 実験データファイル中、実験データを読み出す最初の行の番号。省略時は 1, すなわち先頭行から読み出します。

- reference_last_line

  形式: 整数型 (default: 実験データファイルの最後の行の行数)

  説明: 実験データファイル中、実験データを読み出す最後の行の番号。省略時は最終行まで読み出します。

- exp_number

  形式: 整数型のリスト

  説明: 実験データファイル中、実験データとして読み出す列番号。複数指定が可能。設定した値は、前述の cal_number （[config] セクション）の値とリストの長さを一致させる必要があります。

4.2 ソルバー用補助ファイル

4.2.1 入力テンプレートファイル

入力テンプレートファイル template.txt は surf.exe の入力ファイルを作成するためのテンプレートです。動かすパラメータ（求めたい原子座標などの値）を「 value_* 」などの適当な文字列に置き換えます。使用する文字列は入力ファイルの [solver] - [param] セクションにある、string_list で指定します。以下、テンプレートの例を記載します。

```
2                                   ,NELMS,  -------- Ge(001)-c4x2
32,1.2,0.15                         ,Ge Z,da1,sap
0.6,0.6,0.6                         ,BH(I),BK(I),BZ(I)
32,1.2,0.15                         ,Ge Z,da1,sap
0.4,0.4,0.4                         ,BH(I),BK(I),BZ(I)
9,4,0,0,2,1.7,-0.5,0.5              ,NSGS,msa,msb,nsa,nsb,dthick,DXS,DYS
8                                   ,NATM
1, 1.0,  value_01,  1.00000,  5.231000   ,IELM(I),ocr(I),X(I),Y(I),Z(I
1, 1.0,  value_02,  1.00000,  4.371000
2, 1.0,  1.50000,  1.50000,  3.596000
2, 1.0,  2.00000,  1.49751,  2.100000
2, 1.0,  1.00000,  1.50000,  2.000000
2, 1.0,  0.00000,  1.00000,  0.849425
2, 1.0,  2.00000,  1.00000,  0.809425
2, 1.0,  1.00997,  1.00000,  0.599425
1,1                                 ,(WDOM,I=1,NDOM)
```

この場合、 value_01, value_02 が動かすパラメータとなります。

4.2.2 ターゲット参照ファイル

ターゲットにするデータが格納されたファイル experiment.txt を指定します。第一列に角度、第二列以降に反射強度にコンボリューションを計算した値が入ってます。以下、ファイルの例を示します。

```
3.00000e-01 8.17149e-03 1.03057e-05 8.88164e-15 ...
4.00000e-01 1.13871e-02 4.01611e-05 2.23952e-13 ...
5.00000e-01 1.44044e-02 1.29668e-04 4.53633e-12 ...
6.00000e-01 1.68659e-02 3.49471e-04 7.38656e-11 ...
7.00000e-01 1.85375e-02 7.93037e-04 9.67719e-10 ...
8.00000e-01 1.93113e-02 1.52987e-03 1.02117e-08 ...
9.00000e-01 1.92590e-02 2.53448e-03 8.69136e-08 ...
1.00000e+00 1.86780e-02 3.64176e-03 5.97661e-07 ...
1.10000e+00 1.80255e-02 4.57932e-03 3.32760e-06 ...
1.20000e+00 1.77339e-02 5.07634e-03 1.50410e-05 ...
1.30000e+00 1.80264e-02 4.99008e-03 5.53791e-05 ...
...
```

4.2.3 出力ファイル

sim-trhepd-rheed では、 surf.exe で出力されるファイルが、ランクの番号が記載されたフォルダ下にある Log%%%%%_##### フォルダに一式出力されます。%%%%% はアルゴリズムの反復回数 step (例：モンテカルロステップ数) で、##### はアルゴリズムにおけるグループの番号 set (例：モンテカルロにおけるレプリカ番号) です。大規模計算ではこれらのフォルダの数が多くなり、時には計算機のストレージの制限に引っかかることがあります。そのような場合には、 solver.post.remove_work_dir パラメータを true にして、計算が終了した作業フォルダを削除してください。以下では、 odatse-STR で独自に出力するファイルについて説明します。

4.2.4 stdout

surf.exe が出力する標準出力が記載されています。

以下、出力例です。

```
bulk-filename (end=e) ? :
bulkP.b
structure-filename (end=e) ? :
surf.txt
output-filename :
surf-bulkP.s
```

4.2.5 RockingCurve_calculated.txt

generate_rocking_curve ([solver] セクション) が"true"の場合のみ Log%%%%%_##### フォルダに出力されます。

ファイル冒頭、 # で始まる行はヘッダーです。ヘッダーには探索変数の値、目的関数値 f(x) オプションで指定した Rfactor_type, normalization, weight_type, cal_number, オプションで指定またはプログラムが計算したデータ列ごとの重み spot_weight, データ部分のどの列に何が記されているか (例： # #0 glanceing_angle など) が記されています。

が付いていない部分はデータ表記部分になります。1 列目は視写角、2 列目以降はデータ列ごとに強度が記しています。どのデータ列が記されているかはヘッダーの表記で確認できます。例えば

```
# #0 glancing_angle
# #1 cal_number=1
# #2 cal_number=2
# #3 cal_number=4
```

との記載があれば、1 列目は視写角、2 列目は計算データファイルの 1 列目に相当する反射強度、3 列目は計算データファイルの 2 列目に相当する反射強度、4 列目は計算データファイルの 4 列目に相当する反射強度が記されていることがわかります。

また、各列の反射強度は各列の総和が 1 になるように規格化されています。目的関数値 (R-factor 及び R-factor の二乗) を算出する際は、データ列ごとの重み spot_weight を加味して計算されています。

以下、出力例です。

```
#value_01 =  0.00000 value_02 =  0.00000
#Rfactor_type = A
#normalization = MANY_BEAM
#weight_type = manual
#fx(x) = 0.036861804623405O5
#cal_number = [1, 2, 4, 6, 8]
#spot_weight = [0.933 0.026 0.036 0.003 0.002]
#NOTICE : Intensities are NOT multiplied by spot_weight.
#The intensity I_(spot) for each spot is normalized as in the following equation.
```

(次のページに続く)

(前のページからの続き)

```
#sum( I_(spot) ) = 1
#
# #0 glancing_angle
# #1 cal_number=1
# #2 cal_number=2
# #3 cal_number=4
# #4 cal_number=6
# #5 cal_number=8
0.30000 1.278160358686800e-02 1.378767858296659e-04 8.396046839668212e-14 1.
↪342648818357391e-30 6.697979700048016e-53
0.40000 1.778953628930054e-02 5.281839702773564e-04 2.108814173486245e-12 2.
↪467220122612354e-28 7.252675318478533e-50
0.50000 2.247181148723425e-02 1.671115124520428e-03 4.250758278908295e-11 3.
↪632860054842994e-26 6.291667506376419e-47
...
```

第5章　謝辞

本ソフトウェアは、東京大学物性研究所 ソフトウェア開発・高度化プロジェクト (2020, 2021, 2024 年度) の支援を受け開発されました。

順問題ソルバーの実装にあたり、花田貴氏 (東北大学)、望月出海氏 (KEK) にお世話になりました。この場を借りて感謝します。

第6章　お問い合わせ

odatse-STR に関するお問い合わせはこちらにお寄せください。

- バグ報告

  odatse-STR のバグ関連の報告は GitHub の Issues で受け付けています。

  バグを早期に解決するため、報告時には次のガイドラインに従ってください。

  - 使用している ODAT-SE および odatse-STR のバージョンを指定してください。
  - インストールに問題がある場合には、使用しているオペレーティングシステムとコンパイラの情報についてお知らせください。
  - 実行に問題が生じた場合は、実行に使用した入力ファイルとその出力を記載してください。

- その他

  研究に関連するトピックなど GitHub の Issues で相談しづらいことを問い合わせる際には、以下の連絡先にコンタクトをしてください。

  E-mail: `2dmat-dev__at__issp.u-tokyo.ac.jp` (_at_を@に変更してください)